

BÀI 9: LỆNH TRUY VẤN DỮ LIỆU LỒNG NHAU

Mục tiêu cần đạt được của bài dạy về kiến thức, kỹ năng

Về kiến thức:

- + Hiểu được khái niệm lệnh truy vấn dữ liệu lồng nhau.
- + Hiểu được cú pháp của các trường hợp sử dụng lệnh truy vấn dữ liệu lồng nhau

Về kỹ năng:

- + Vận dụng lệnh truy vấn lồng nhau vào giải quyết các bài toán đã cho.

9.1. Khái niệm

Lệnh truy vấn dữ liệu lồng nhau (Nested Queries), còn được gọi là **subqueries**, là một truy vấn SQL được nhúng bên trong một truy vấn SQL khác.

Mục đích: Truy vấn dữ liệu lồng nhau cho phép chúng ta thực hiện các phép toán phức tạp, so sánh hoặc lọc dữ liệu dựa trên kết quả của một truy vấn khác.

Truy vấn con (Subquery) là một câu lệnh SELECT được lồng vào bên trong một câu lệnh SELECT, INSERT, UPDATE, DELETE hoặc bên trong một truy vấn con khác. Loại truy vấn này được sử dụng để biểu diễn cho những truy vấn trong đó điều kiện truy vấn dữ liệu cần phải sử dụng đến kết quả của một truy vấn khác.

9.2. Cú pháp

9.2.1. Truy vấn con trong mệnh đề SELECT

```
SELECT column1, column2,  
      (SELECT aggregate_function(column3)  
      FROM table2  
      WHERE table2.column1 = table1.column1) AS alias_name  
FROM table1;
```

Giải thích: Truy vấn con nằm trong mệnh đề SELECT có thể trả về một giá trị đơn lẻ cho mỗi dòng của truy vấn chính.

Ví dụ: Tìm tên hàng và số lượng của hàng đó trong hóa đơn có số lượng nhiều nhất.

```
SELECT h.TenH, (SELECT TOP 1 SoLuong  
               FROM ChiTietHoaDon cthd  
               WHERE cthd.MaH = h.MaH  
               ORDER BY cthd.SoLuong DESC) AS MaxQuantity  
FROM Hang h;
```

Giải thích: Truy vấn này lấy tên hàng và số lượng hàng đó trong hóa đơn mà số lượng của hàng đó lớn nhất. **Truy vấn con** trong mệnh đề SELECT tìm số lượng lớn nhất cho từng hàng.

9.2.2. Truy vấn con trong mệnh đề WHERE

```
SELECT column1, column2
```

```

FROM table1
WHERE column3 = (SELECT column3
                  FROM table2
                  WHERE condition);

```

Giải thích: Truy vấn con trong mệnh đề WHERE giúp lọc các bản ghi trong bảng chính dựa trên kết quả của Truy vấn con

Ví dụ: Tìm tất cả các hóa đơn mà tổng số lượng hàng trong hóa đơn lớn hơn số lượng trung bình của tất cả các hóa đơn.

```

SELECT SoHD
FROM HoaDon
WHERE SoHD IN
    (SELECT SoHD
     FROM ChiTietHoaDon
     GROUP BY SoHD
     HAVING SUM(SoLuong) >
        ( SELECT AVG(TotalQuantity)
          FROM ( SELECT SoHD, SUM(SoLuong) AS TotalQuantity
                  FROM ChiTietHoaDon
                  GROUP BY SoHD
                ) AS AverageQuantities
        ));

```

Giải thích: Truy vấn con trong mệnh đề WHERE tính tổng số lượng hàng cho từng hóa đơn và so sánh với số lượng trung bình của tất cả các hóa đơn.

9.2.3. Truy vấn con trong mệnh đề FROM

```

SELECT alias_name.column1, alias_name.column2
FROM (SELECT column1, column2
      FROM table2
      WHERE condition) AS alias_name;

```

Giải thích: Truy vấn con trong mệnh đề FROM tạo ra một bảng tạm, và các cột của bảng tạm này có thể được truy cập trong câu lệnh chính.

Ví dụ: Tìm các khách hàng có số lượng hóa đơn nhiều hơn số lượng trung bình của tất cả khách hàng.

```

SELECT k.TenK, order_counts.OrderCount
FROM Khach k
JOIN ( SELECT MaK, COUNT(*) AS OrderCount

```

```

FROM HoaDon
GROUP BY MaK
) AS order_counts ON k.MaK = order_counts.MaK
WHERE order_counts.OrderCount > (
    SELECT AVG(OrderCount)
    FROM (SELECT MaK, COUNT(*) AS OrderCount
          FROM HoaDon
          GROUP BY MaK
    ) AS avg_orders
);

```

Giải thích: Truy vấn con trong mệnh đề FROM tính số lượng hóa đơn cho từng khách hàng. Sau đó, so sánh số lượng hóa đơn của từng khách hàng với số lượng trung bình của tất cả khách hàng.

9.2.4. Truy vấn con trong mệnh đề HAVING

```

SELECT column1, COUNT(*)
FROM table1
GROUP BY column1
HAVING COUNT(*) > (SELECT AVG(count_column)
                   FROM table2
                   WHERE condition);

```

Giải thích: Truy vấn con trong mệnh đề HAVING cho phép lọc các nhóm dữ liệu dựa trên điều kiện tổng hợp.

Ví dụ: Tìm các hàng hóa có số lượng trung bình trong hóa đơn lớn hơn số lượng trung bình của tất cả các hàng hóa.

```

SELECT MaH, TenH
FROM Hang
WHERE (SELECT AVG(SoLuong)
      FROM ChiTietHoaDon
      WHERE MaH = Hang.MaH) > (SELECT AVG(SoLuong)
                                FROM Hang);

```

Giải thích:

1. Truy vấn con 1 (AVG trong ChiTietHoaDon):

- **SELECT AVG(SoLuong) FROM ChiTietHoaDon WHERE MaH = Hang.MaH:** Truy vấn con này tính toán số lượng trung bình (AVG(SoLuong)) của hàng hóa trong các hóa đơn cho mỗi mã hàng (MaH) trong bảng **Hang**. Đây là số lượng trung bình của hàng hóa trong các hóa đơn.

2. Truy vấn con 2 (AVG trong Hang):

- **SELECT AVG(SoLuong) FROM Hang:** Truy vấn con này tính toán số lượng trung bình của tất cả các hàng hóa trong bảng **Hang**.

3. So Sánh:

- Truy vấn con 1 phải lớn hơn Truy vấn con 2 để chọn các hàng hóa mà số lượng trung bình trong hóa đơn của chúng lớn hơn số lượng trung bình của tất cả các hàng hóa.

Giả sử chúng ta có các bảng với dữ liệu như sau:

Bảng Hang:

MaH	TenH	DVT	SoLuong
1	Hàng A	Cái	100
2	Hàng B	Cái	200
3	Hàng C	Cái	150

Bảng ChiTietHoaDon:

SoHD	MaH	SoLuong
1	1	120
1	2	210
2	1	80
2	3	160

Kết quả của câu lệnh truy vấn trên sẽ là các hàng hóa có số lượng trung bình trong các hóa đơn lớn hơn số lượng trung bình của tất cả các hàng hóa trong bảng **Hang**.

9.2.5. Truy vấn con khi sử dụng toán tử IN

Toán tử **IN** trong SQL được sử dụng để so sánh một giá trị với một tập hợp các giá trị trả về từ một truy vấn con (subquery). Nếu giá trị so sánh nằm trong tập hợp, điều kiện sẽ được thỏa mãn.

Toán tử **IN** cùng với subquery giúp lọc dữ liệu dựa trên một tập hợp giá trị được trả về từ một truy vấn con khác.

Cú pháp:

WHERE <biểu thức> **[NOT] IN** (câu truy vấn con)

Ví dụ: Tìm các hàng hóa mà mã hàng có xuất hiện trong bảng ChiTietHoaDon, nghĩa là các hàng hóa đã được ghi nhận trong ít nhất một hóa đơn.

```
SELECT MaH, TenH, DVT, SoLuong
FROM Hang
```

WHERE MaH IN (SELECT DISTINCT MaH FROM ChiTietHoaDon);

Giải Thích

1. **Truy vấn con: SELECT DISTINCT MaH FROM ChiTietHoaDon:** Truy vấn con này trả về danh sách các mã hàng (MaH) đã xuất hiện trong bảng ChiTietHoaDon. Sử dụng **DISTINCT** để loại bỏ các mã hàng trùng lặp, vì một mã hàng có thể xuất hiện nhiều lần trong bảng ChiTietHoaDon.
2. **Toán Tử IN: WHERE MaH IN (...):** Toán tử **IN** so sánh giá trị MaH trong bảng Hang với danh sách các mã hàng trả về từ subquery. Nếu mã hàng có trong danh sách, hàng đó sẽ được chọn.

Giả sử dữ liệu trong các bảng là như sau:

Bảng Hang:

MaH	TenH	DVT	SoLuong
1	Hàng A	Cái	100
2	Hàng B	Cái	200
3	Hàng C	Cái	150
4	Hàng D	Cái	300

Bảng ChiTietHoaDon:

SoHD	MaH	SoLuong	DonGia
1	1	10	100
1	2	5	200
2	1	7	100
3	3	15	150

Kết quả của câu lệnh SQL trên sẽ là:

MaH	TenH	DVT	SoLuong
1	Hàng A	Cái	100
2	Hàng B	Cái	200
3	Hàng C	Cái	150

Hàng hóa có mã hàng 4 không xuất hiện trong bảng ChiTietHoaDon, vì vậy nó không được đưa vào kết quả.

Trong ví dụ này, nó giúp chúng ta tìm tất cả các hàng hóa đã xuất hiện trong ít nhất một hóa đơn.

9.2.6. Truy vấn con khi sử dụng toán tử ANY

Toán tử **ANY** (hoặc **SOME** trong một số hệ quản trị cơ sở dữ liệu) được sử dụng để so sánh một giá trị với tất cả các giá trị trả về từ một truy vấn con. **ANY** kiểm tra điều kiện cho ít nhất một giá trị trong tập hợp kết quả của truy vấn con.

Điều kiện	Kết quả tương đương	Mô tả
$X > ANY(1,2)$	$X > 1$	X phải lớn hơn ít nhất 1 giá trị trả về từ câu truy vấn con
$X < ANY(1,2)$	$X < 2$	X phải nhỏ hơn ít nhất 1 giá trị trả về từ câu truy vấn con
$X = ANY(1,2)$	$(X=1) \text{ or } (X=2)$	$X \text{ IN}(1,2)$

Ví dụ: Tìm các hàng hóa có số lượng tồn kho lớn hơn số lượng phân phối tối thiểu trong bất kỳ hóa đơn nào.

SELECT MaH, TenH, SoLuong

FROM Hang

WHERE SoLuong > **ANY** (**SELECT** SoLuong **FROM** ChiTietHoaDon);

Giải Thích

1. **Subquery:**

- **SELECT SoLuong FROM ChiTietHoaDon:** Truy vấn con này trả về tất cả các giá trị số lượng phân phối (SoLuong) từ bảng ChiTietHoaDon.

2. **Toán Tử ANY:**

- **WHERE SoLuong > ANY (...):** Toán tử **ANY** kiểm tra xem số lượng tồn kho của hàng hóa (SoLuong trong bảng Hang) có lớn hơn ít nhất một giá trị số lượng phân phối từ **Truy vấn con** không. Nếu đúng, hàng hóa đó sẽ được chọn.

Ví Dụ Cụ Thể

Giả sử dữ liệu trong các bảng như sau:

Bảng Hang:

MaH	TenH	DVT	SoLuong
1	Hàng A	Cái	100
2	Hàng B	Cái	50
3	Hàng C	Cái	200
4	Hàng D	Cái	30

Bảng ChiTietHoaDon:

SoHD	MaH	SoLuong	DonGia
1	1	80	100
1	2	60	200
2	3	150	150
3	4	25	300

Kết quả:

- **Hàng A:** Số lượng tồn kho là 100, lớn hơn 80 (số lượng trong ChiTietHoaDon).
- **Hàng B:** Số lượng tồn kho là 50, không lớn hơn bất kỳ giá trị nào trong ChiTietHoaDon.
- **Hàng C:** Số lượng tồn kho là 200, lớn hơn 150 (số lượng trong ChiTietHoaDon).
- **Hàng D:** Số lượng tồn kho là 30, lớn hơn 25 (số lượng trong ChiTietHoaDon).

Kết quả của câu lệnh SQL sẽ là:

MaH	TenH	SoLuong
1	Hàng A	100
3	Hàng C	200
4	Hàng D	30

Toán tử **ANY** là một công cụ mạnh mẽ để so sánh một giá trị với nhiều giá trị trong tập hợp. Trong ví dụ này, nó giúp xác định các hàng hóa có số lượng tồn kho lớn hơn ít nhất một trong các số lượng phân phối ghi nhận trong bảng

9.2.7. Truy vấn con khi sử dụng toán tử **ALL**

1. Khái niệm

Toán tử **ALL** trong SQL được sử dụng để so sánh một giá trị với **tất cả** các giá trị trả về từ một **Truy vấn con**. Nó thường được sử dụng kết hợp với các toán tử so sánh như =, >, <, >=, <=, và <> (khác).

Toán tử **ALL** yêu cầu rằng so sánh phải đúng cho tất cả các giá trị được trả về bởi truy vấn con.

Toán tử **ALL** trong SQL là công cụ mạnh để kiểm tra điều kiện trên toàn bộ tập giá trị trả về từ truy vấn con. Nó hữu ích trong các tình huống cần so sánh một giá trị với toàn bộ các giá trị trong một danh sách con và đòi hỏi điều kiện phải thỏa mãn với tất cả các giá trị trong danh sách đó.

2. Cú pháp

```
SELECT column_name(s)
```

```
FROM table_name
```

WHERE expression operator ALL (subquery);

Trong đó:

- **expression:** Biểu thức hoặc cột cần so sánh.
- **operator:** Một toán tử so sánh (>, <, =, >=, <=, <>).
- **subquery:** Một truy vấn con trả về một tập hợp giá trị.

3. Ý nghĩa và hoạt động của toán tử ALL

Toán tử **ALL** cho phép so sánh một giá trị với **tất cả** các giá trị được trả về từ truy vấn con.

- Nếu sử dụng với toán tử >: Biểu thức phải lớn hơn **tất cả** các giá trị trong truy vấn con.
- Nếu sử dụng với toán tử <: Biểu thức phải nhỏ hơn **tất cả** các giá trị trong truy vấn con.
- Nếu sử dụng với =: Biểu thức phải bằng **tất cả** các giá trị trong truy vấn con (điều này ít được sử dụng vì chỉ khi tất cả các giá trị giống nhau mới có thể trả về kết quả).

4. Ví dụ

Ví dụ 1: So sánh với toán tử > và ALL

Yêu cầu: Tìm các sản phẩm có giá lớn hơn tất cả các sản phẩm có trong danh mục hàng hóa của một nhà cung cấp cụ thể.

SELECT TenSP, Gia

FROM SANPHAM

WHERE Gia > ALL (

SELECT Gia

FROM SANPHAM

WHERE NhaCungCap = 'NC01'

);

- **Giải thích:** Truy vấn này trả về tên và giá của các sản phẩm có giá **lớn hơn tất cả** các sản phẩm từ nhà cung cấp 'NC01'. Nếu có một sản phẩm có giá lớn hơn tất cả các sản phẩm của nhà cung cấp này, nó sẽ được liệt kê.

Ví dụ 2: Sử dụng toán tử <= và ALL

Yêu cầu: Tìm các nhân viên có mức lương nhỏ hơn hoặc bằng tất cả các mức lương của nhân viên thuộc phòng ban '**Phòng Kinh Doanh**'.

SELECT MaNV, HoTen, Luong

FROM NHANVIEN

WHERE Luong <= ALL (

SELECT Luong

FROM NHANVIEN

WHERE PhongBan = 'Phòng Kinh Doanh'

);

- **Giải thích:** Truy vấn này trả về mã, tên và lương của các nhân viên có mức lương **nhỏ hơn hoặc bằng tất cả** các nhân viên thuộc phòng ban '**Phòng Kinh Doanh**'.

5. So sánh toán tử **ALL** với **ANY**

Toán tử **ALL** yêu cầu rằng biểu thức phải thỏa mãn điều kiện so sánh với **tất cả** các giá trị trả về từ truy vấn con, trong khi toán tử **ANY** chỉ yêu cầu điều kiện đúng với **ít nhất một** giá trị trong tập kết quả của truy vấn con.

- **ALL:** Điều kiện phải đúng với **tất cả** các giá trị.
- **ANY:** Điều kiện chỉ cần đúng với **một hoặc nhiều** giá trị.

Ví dụ so sánh:

-- Sử dụng ALL

SELECT MaNV, HoTen, Luong

FROM NHANVIEN

WHERE Luong > **ALL** (

SELECT Luong

FROM NHANVIEN

WHERE PhongBan = 'Phòng Kế Toán'

);

-- Sử dụng ANY

SELECT MaNV, HoTen, Luong

FROM NHANVIEN

WHERE Luong > **ANY** (

SELECT Luong

FROM NHANVIEN

WHERE PhongBan = 'Phòng Kế Toán'

);

- Với **ALL**: Chỉ trả về nhân viên có mức lương lớn hơn tất cả nhân viên trong phòng kế toán.
- Với **ANY**: Trả về nhân viên có mức lương lớn hơn **ít nhất một** nhân viên trong phòng kế toán.

6. Chú ý

- Truy vấn con được sử dụng với toán tử **ALL** phải trả về một cột (single column).
- Toán tử **ALL** được sử dụng để kiểm tra điều kiện với **tất cả** các giá trị trong một tập hợp, và kết quả chỉ trả về nếu điều kiện đúng với tất cả giá trị trong tập.

9.2.8. Truy vấn con khi Sử dụng toán tử EXISTS

Toán tử **EXISTS** được sử dụng trong SQL để kiểm tra sự tồn tại của các bản ghi trong kết quả của một truy vấn con. Nếu truy vấn con trả về ít nhất một bản ghi, thì **EXISTS** sẽ trả về TRUE; nếu không có bản ghi nào được trả về, thì sẽ trả về FALSE.

CÚ PHÁP:

WHERE [NOT] EXISTS (câu truy vấn con)

Ví dụ: Tìm các hàng hóa mà đã có ít nhất một bản ghi trong bảng ChiTietHoaDon, nghĩa là hàng hóa đó đã xuất hiện trong ít nhất một hóa đơn.

SELECT MaH, TenH, DVT, SoLuong

FROM Hang H

WHERE EXISTS (SELECT 1 FROM ChiTietHoaDon CTHD
WHERE H.MaH = CTHD.MaH);

Giải Thích

1. Truy vấn con:

- **SELECT 1 FROM ChiTietHoaDon CTHD WHERE H.MaH = CTHD.MaH:** Truy vấn con này kiểm tra xem có bất kỳ bản ghi nào trong bảng ChiTietHoaDon với mã hàng (MaH) trùng với mã hàng trong bảng Hang. Chúng ta chỉ cần trả về 1 để kiểm tra sự tồn tại của bản ghi, không cần dữ liệu cụ thể.

2. Toán Tử EXISTS:

- **WHERE EXISTS (...):** Toán tử **EXISTS** kiểm tra nếu **Truy vấn con** trả về ít nhất một bản ghi. Nếu có, điều kiện được thỏa mãn và bản ghi từ bảng Hang sẽ được chọn.

Ví Dụ Cụ Thể

Giả sử dữ liệu trong các bảng là như sau:

Bảng Hang:

MaH	TenH	DVT	SoLuong
1	Hàng A	Cái	100
2	Hàng B	Cái	50
3	Hàng C	Cái	200
4	Hàng D	Cái	30

Bảng ChiTietHoaDon:

SoHD	MaH	SoLuong	DonGia
1	1	80	100

1	2	60	200
2	3	150	150

Kết quả của câu lệnh SQL sẽ là:

MaH	TenH	DVT	SoLuong
1	Hàng A	Cái	100
2	Hàng B	Cái	50
3	Hàng C	Cái	200

Hàng hóa có mã hàng 4 không xuất hiện trong bảng ChiTietHoaDon, vì vậy nó không được đưa vào kết quả.

Toán tử **EXISTS** rất hữu ích khi bạn muốn kiểm tra sự tồn tại của các bản ghi trong một tập hợp dữ liệu mà không cần phải biết chi tiết cụ thể về bản ghi đó. Trong ví dụ này, nó giúp xác định các hàng hóa đã xuất hiện trong ít nhất một hóa đơn.

9.3. Các truy vấn con phải tuân theo các quy tắc sau:

- Có thể đặt câu truy vấn con trong các mệnh đề WHERE, HAVING và FROM của câu truy vấn cha (câu truy vấn bên ngoài).
- Truy vấn con có thể được sử dụng với các câu lệnh SELECT, UPDATE, INSERT, DELETE cùng với toán tử biểu thức. Nó có thể là toán tử bình đẳng hoặc toán tử so sánh như =, >, <, <= và toán tử Like.
- Các truy vấn con phải được bao trong các dấu ngoặc đơn ().
- Câu lệnh ORDER BY không thể được sử dụng trong một truy vấn con, mặc dù truy vấn chính có thể sử dụng một ORDER BY.

GROUP BY có thể được sử dụng để thực hiện cùng tính năng như ORDER BY trong một truy vấn con.

- Các truy vấn con, mà trả về nhiều hơn một hàng, có thể chỉ được sử dụng với các toán tử nhân, ví dụ: toán tử IN.
- Danh sách của SELECT không thể bao gồm bất kỳ tham chiếu nào tới các giá trị mà ước lượng một BLOB, ARRAY, CLOB hoặc NCLOB.
- Một truy vấn con không thể bị bao quanh một cách trực tiếp trong một hàm tập hợp.
- Toán tử BETWEEN không thể được sử dụng với một truy vấn con; tuy nhiên, toán tử BETWEEN có thể được sử dụng bên trong truy vấn con.

9.4. Lưu ý khi sử dụng Subquery

1. **Hiệu Suất:** Các **Truy vấn con** có thể ảnh hưởng đến hiệu suất truy vấn, đặc biệt là khi **Truy vấn con** trả về nhiều dữ liệu hoặc được sử dụng nhiều lần. Sử dụng chỉ khi cần thiết và xem xét các cách tối ưu hóa nếu cần.

2. **Subquery so với Joins:** Trong một số trường hợp, Các **Truy vấn con** có thể được thay thế bằng các phép nối (JOINS). Cần cân nhắc kỹ lưỡng để chọn phương pháp phù hợp với yêu cầu.
3. **Subquery trả về nhiều giá trị:** Nếu **Truy vấn con** trả về nhiều giá trị, cần sử dụng các toán tử thích hợp như **IN** hoặc **EXISTS**.

So sánh join và câu lệnh truy vấn con

JOIN	CÂU LỆNH TRUY VẤN CON
Kết quả có thể bao gồm các cột của cả 2 bảng	Không thể bao gồm các cột của câu truy vấn con
Sử dụng mối quan hệ giữa 2 bảng	
Chạy nhanh hơn	
	Có thể chuyển 1 giá trị tính toán ra câu lệnh truy vấn bên ngoài
	Dễ viết code và dễ hiểu

9.5 Ví dụ cụ thể

Với CSDL quản lý bán hàng. Hãy thực hiện các yêu cầu sau:

1: Đưa ra tên các khách hàng chưa từng mua mặt hàng nào

```
SELECT TenK
FROM Khách
WHERE Khách.MaK not in (SELECT HoaDon.MaK
FROM HoaDon)
```

2: Đưa ra tên các mặt hàng chưa được khách hàng nào mua

```
SELECT TenH
FROM Hang
WHERE Hang.MaH not in (SELECT ChiTietHoaDon.MaH
FROM ChiTietHoaDon)
```

3: Câu lệnh sau liệt kê thông tin các khách hàng đã mua hàng tại cửa hàng

```
SELECT Khách.*
FROM Khách
WHERE Khách.MaK IN (SELECT HoaDon.MaK
FROM HoaDon)
```

Hoặc có thể viết như sau:

```
SELECT Khach.*  
FROM Khach, HoaDon  
WHERE Khach.MaK = HoaDon.MaK
```

4: Tìm những khách hàng đã từng mua mặt hàng có tên hàng là ‘Máy tính’

```
SELECT Khach.*  
FROM Khach  
WHERE Khach.MaK IN (SELECT HoaDon.MaK  
                     FROM HoaDon, ChiTietHoaDon  
                     WHERE HoaDon.SoHD = ChiTietHoaDon.SoHD and  
                           ChiTietHoaDon.MaH IN (SELECT Hang.MaH  
                                                  FROM Hang  
                                                  WHERE Hang.TenH = N‘Máy tính’))
```

5: Tìm những khách hàng đã mua hàng nhưng chưa từng mua mặt hàng có mã hàng là ‘H1’

```
SELECT Khach.*  
FROM Khach, HoaDon  
WHERE Khach.MaK = HoaDon.MaK  
AND HoaDon.SoHD NOT IN (SELECT ChiTietHoaDon.SoHD  
                        FROM ChiTietHoaDon  
                        WHERE ChiTietHoaDon.MaH = ‘H1’)
```

Câu hỏi

Câu 1. Trình bày khái niệm về truy vấn dữ liệu lồng nhau? Cho ví dụ minh họa?

Câu 2. Trình bày các quy tắc khi sử dụng truy vấn dữ liệu lồng nhau?